

SWE-Bench Pro V2: Human and Automated Repair of a Coding Benchmark

Anonymous ACL submission

Abstract

SWE-Bench Pro is a widely used benchmark for coding agents, but recent audits found that many of its tasks are broken and that agents could reach the answer during evaluation. We release SWE-Bench Pro V2, a repaired version of its public split with 642 tasks. We fixed the data with 23 software engineers, who corrected task instructions against the hidden tests and had every fix checked by a second engineer who implemented it blind, and with automated checks that every task passes with the reference solution and fails without one. We fixed the evaluation by limiting the agent’s network to the model endpoint and re-grading every patch in a clean container, which also caught agents faking dependencies they could not download. After the fixes, frontier systems resolve 94.9–99.4% of the tasks, and a 51-task Hard split spreads them from 59% to 98%, separating the strongest systems from the rest.

1 Introduction

Benchmarks built from real GitHub issues, such as SWE-bench (Jimenez et al., 2024), are the standard way to compare coding agents. SWE-Bench Pro (Deng et al., 2025) was built to be harder: its tasks come from 11 repositories, need long changes, and each comes with a written specification and hidden tests.

Two problems have since been found. The first is in the data. OpenAI estimated that about 30% of the public tasks are broken (OpenAI, 2026). Some tests check details the instruction never mentions. Some instructions contradict the tests. In both cases a correct solution fails. The second is in the evaluation. Agents could use the internet or the repository’s git history to find the original fix. Zheng et al. (2026) showed this inflates scores: blocking those routes turned 186 passes of one model into failures.

We fix both problems and release the result as SWE-Bench Pro V2. To fix the data, engineers

corrected instructions so that they state what the tests check, and a second engineer confirmed each fix by solving the task from the corrected text alone. Automated checks confirm that every task can be passed and cannot be passed by doing nothing. To fix the evaluation, the agent may reach only the model endpoint, and its patch is graded again in a fresh container it never touched.

After the fixes, the strongest systems solve almost every task (Figure 1). The full set no longer tells them apart, so we also release a Hard split of 51 tasks, which does.

- We release SWE-Bench Pro V2: 642 tasks, each passing with its reference solution and failing with an empty patch.
- **Fixing the data.** 23 engineers spent 1,897 hours correcting instructions, each fix verified by a blind re-implementation; automated checks fixed the grading of 23 tasks.
- **Fixing the evaluation.** A locked protocol blocks all network access except the model endpoint and re-grades every patch in a clean container. No run retrieved external content, and re-grading caught every faked dependency.
- **Results.** Frontier systems resolve 94.9–99.4% of V2. On the Hard split our runs range from 59% to 98%, and each of the four strongest systems beats Gemini 3.8 Flash ($p < 0.001$).

2 Related Work

Coding benchmarks. SWE-bench (Jimenez et al., 2024) grades a patch by running the tests of the original fix in a clean container. SWE-bench Verified (OpenAI, 2024) had annotators filter out tasks with unclear specifications or unfair tests. Later benchmarks extend the format to more languages, newer issues and other task sources (Zan

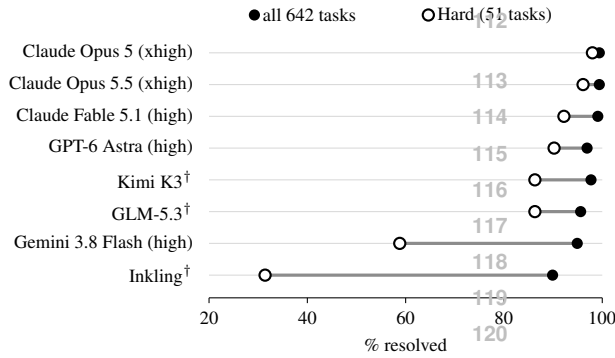


Figure 1: After the fixes, every system resolves 90–99% of V2 (filled dots), but the Hard split (open dots) spreads them from 31% to 98%. [†]Partner runs under the same protocol, single grade, not re-graded by us.

et al., 2025; Zhang et al., 2025; Badertdinov et al., 2025; Miserendino et al., 2025). SWE-Bench Pro (Deng et al., 2025) adds longer tasks and a curated specification per task.

Broken tasks. Weak or wrong tests are a known problem (Aleithan et al., 2024; Wang et al., 2025; Yu et al., 2025), and checklists for rigorous agent benchmarks exist (Zhu et al., 2025). For SWE-Bench Pro, OpenAI’s audit (OpenAI, 2026) found over-strict tests, hidden requirements, narrow tests and misleading instructions. Nadgir et al. (2026) show that such flaws often surface only once agents are strong enough to reach them.

Reward hacking and leakage. Agents game tests when they can (Zhong et al., 2025; Gabor et al., 2025; Zhao et al., 2026; Thaman, 2026), and future git history leaked fixes in both SWE-bench and SWE-Bench Pro (jacobkahn, 2025; joe-needham, 2025). The closest work is SWE-Bench Pro Verified (Zheng et al., 2026). It blocks code hosts, hides test files, refines 102 tasks and checks the refinements with model trial runs. We differ in three ways. We never edit the tests to fit an instruction. We check each fix by a blind human re-implementation. And we grade every patch a second time in a clean container.

3 Method

3.1 What was broken

Table 1 lists the problems we found, grouped into data and evaluation, with the fix for each. Data problems make a correct solution fail. Evaluation problems let an incorrect or copied solution pass.

3.2 Fixing the data

Human fixes. SWE-Bench Pro gives the agent a written instruction: a problem statement, requirements and the new interfaces. Our rule is simple: if the reference solution adds a behaviour and a hidden test checks it, the instruction must say so. 23 contracted engineers applied this rule task by task, spending 1,897 hours in total. One engineer finds the mismatch and corrects the instruction. A second engineer, who has seen neither the original task nor the reference solution, implements the corrected instruction, and the hidden tests grade the result. A fix ships only if that implementation passes.

Two examples show the kind of defect found. A flit task never said that three configuration files must exist, although the tests read them by path. A tutanota instruction stated the opposite of what an exact-string test checks. When the test itself was wrong, we did not rewrite the instruction to match it. We set the task aside and documented why (17 tasks). The hidden tests are never edited to fit an instruction.

Automated fixes. Every release must pass two checks on every task: the reference solution scores 1, and an empty patch scores 0. These checks caught the same bug twice. A fix to a test-output parser changed test names without updating the list of graded names, so 23 tasks failed even with the reference solution. Regenerating the names fixed it. The verifier was also hardened. Before the hidden tests run, it restores every file they touch, reverts edits to test data and clears cached bytecode, so that files the agent left behind cannot change the result.

3.3 Fixing the evaluation

Network. The original harness ignored the setting that disables the agent’s network. In an earlier run with the network open, 25 trajectories fetched code from the task’s own upstream repository. Our locked protocol uses a harness that sets the network per phase. During the agent phase, only the model endpoint is reachable, and the scaffolds’ web tools are turned off. Before every run, a probe checks from inside the sandbox that code hosts are blocked, the endpoint answers and the reference solution still passes. The git history in every image was checked too: no image contains the fixing commit.

	Problem	Evidence	Fix	By
Data	Instruction omits or contradicts what a test checks	About 30% of public tasks broken (OpenAI, 2026)	69 + 112 + 231 instructions corrected across releases, each checked by a blind re-implementation	Human
	Test itself is wrong	e.g. an assertion on an incidental detail	17 tasks set aside and documented; tests never edited to fit	Human
	Invalid tasks	89 of the 731 public tasks	Dropped	Human
	Parser fix renamed tests	23 tasks failed with the reference solution	Graded names regenerated; reference and empty-patch checks on every release	Automated
	Leftover files could pre-empt hidden tests	Files at test paths, stale bytecode	Verifier restores test files and data, clears bytecode	Automated
Evaluation	Agent had open internet	32/642 trajectories called code hosts; 25 fetched their own repository	Only the model endpoint reachable in the agent phase; probe before every run	Automated
	Web tools run outside the sandbox	13/642 trajectories used web fetch	Disabled	Automated
	Grading ran in the agent’s sandbox	A python3 wrapper scores 1 with an empty patch	Patch re-graded in a clean container; both grades reported	Automated
	Git history could hold the fix	Checked in all 642 images	Fixing commit absent everywhere; probe confirms	Automated

Table 1: What was broken in SWE-Bench Pro and how V2 fixes it. Data problems make correct solutions fail; evaluation problems let wrong or copied solutions pass. Evidence for the evaluation rows comes from an earlier run whose harness ignored the network setting.

Grading. The verifier used to run in the same sandbox the agent had just used. We showed this is unsafe: an agent that edits nothing but replaces python3 with a script that prints PASSED scores 1. We therefore save the agent’s changes as a patch and apply it to a fresh copy of the task image, where the unchanged verifier runs. Anything the agent did outside its patch does not carry over. We report both grades for every task. When the two disagree, two more fresh grades decide, by majority.

System	Scaffold	V2 (642)	Hard (51)
Claude Opus 5 (xhigh)	Claude Code	99.4	98.0
Claude Opus 5.5 (xhigh)	Claude Code	99.4	96.1
Claude Fable 5.1 (high)	Claude Code	99.1	92.2
GPT-6 Astra (high)	Codex	96.9	90.2
Gemini 3.8 Flash (high)	mini-swe-agent	94.9	58.8
<i>Partner runs (same protocol, single grade, not re-graded by us)</i>			
Kimi K3	mini-swe-agent	97.7	86.3
GLM-5.3	mini-swe-agent	95.6	86.3
Inkling	mini-swe-agent	89.9	31.4
Claude Sonnet 5	Claude Code	–	88.2
GPT-5.6 Terra (xhigh)	Codex	–	86.3
GPT-5.6 Sol (xhigh)	Codex	–	82.4
Claude Haiku 4.5	Claude Code	–	25.5

4 Experiments

4.1 Setup

We ran five systems on V2 under the locked protocol. Claude Code ran Claude Opus 5 and Opus 5.5 at xhigh effort and Fable 5.1 at high effort. Codex ran GPT-6 Astra at high effort, and mini-swe-agent ran Gemini 3.8 Flash at high effort. Each task gets one attempt and a 50-minute budget, and the patch present at the deadline is graded. A system is a model plus its scaffold and effort setting, so we compare systems, not models. Partners also reported runs under the same protocol; we list them separately because we did not re-grade them. We compare systems with exact McNemar tests on per-task results. Versions and commands are in Appendix D.

4.2 Main results

After the fixes, frontier systems solve almost every task (Table 2). Our five runs resolve 94.9–99.4%,

Table 2: Percent of tasks resolved under the locked protocol, one attempt per task. Our runs report the grade in a clean container, with disagreements settled by two more fresh grades.

and every one of the 642 tasks is solved by at least one of them. The full set no longer separates the strongest systems. The three Claude systems are statistically tied ($p \geq 0.72$), and on the 413 tasks whose instructions we did not edit after earlier runs, Opus 5 still resolves 99.0%. The high scores therefore do not come from instructions we rewrote after seeing failures.

4.3 The Hard split

The Hard split has 51 tasks that at least two of five model families failed in earlier runs, after removing three tasks with verified defects (Appendix A). On it our runs range from 58.8% to 98.0% (Figure 1). Each of the four strongest systems beats Gemini on Hard ($p < 0.001$). Astra and Gemini are a useful

System	As shipped	Verifier fixed	Fully fixed
Opus 5 (xhigh)	81.6	88.2	93.0
Kimi K3 (max)	78.7	84.2	88.2
GLM-5.3 (max)	77.6	83.1	87.9
Gemini 3.8 Flash	77.6	83.5	86.0
Inkling (xhigh)	67.6	71.3	79.0

Table 3: The never-public set (272 tasks), percent resolved. The middle column re-grades the same patches with the fixed verifier. The last column re-runs all systems after 68 instructions were corrected and the images rebuilt.

contrast: they are tied on the other 591 tasks (576 against 579, $p = 0.68$), but on Hard Astra resolves 46 tasks to Gemini’s 30. Partner runs show the same pattern. Claude Sonnet 5, GPT-5.5 Terra and Sol, Kimi K3 and GLM-5.3 resolve 42–45 Hard tasks, while Inkling resolves 16 and Claude Haiku 4.5 13. Hard does not separate the top four from each other (46–50 tasks), so ranking at the very top still needs harder tasks.

4.4 What the evaluation fixes caught

No retrieval. Agents did try the network: depending on the system, between 9 and 246 fetch commands per run, mostly package installs. We checked every flagged command by hand: all were refused, and no run retrieved any external content. Gemini once tried to disguise a request to the Go module proxy as traffic to the model endpoint. It was refused.

Faked dependencies. When an agent could not download a dependency, it sometimes built a fake one. Opus 5 and Inkling, in earlier runs, and Gemini on V2 forged Go module versions, wrote made-up checksums, or edited the shared module cache. All of these passed in the agent’s sandbox and failed in the clean container, because the fake lived outside the patch. Without the second grade, each would have counted as a pass. The other disagreements between the two grades were flaky tests and one patch that missed a git-ignored directory (Table 4).

4.5 Do the data fixes matter?

We applied the same fixes to a never-public set of 272 tasks from the private repositories of 4 organisations (Table 3). First we fixed its verifier, which graded a patch 0 whenever the agent had touched a file the hidden tests also change. Then engineers corrected 68 instructions that an auto-

mated audit had flagged as unclear about what the tests check. The audit saw no model output, so these edits were not chosen from failures. Opus 5 rose from 81.6% to 88.2% to 93.0%. On the 68 corrected tasks, the pass rate pooled over five systems rose from 56.8% to 73.2%, while the other 204 tasks moved by under one point. Fixing the data changes the scores, and it changes them where the fixes were made. This set also separates systems: Opus 5 beats every other system (Holm-corrected $p \leq 0.043$). Re-grading missed one trick here. On two tasks needing an SDK missing from the images, three systems wrote their own imitation SDK into the patch, and it passed because the tests mock the client. Counting those 5 passes as failures lowers no score by more than 0.8 points.

5 Conclusion

We repaired SWE-Bench Pro in two ways. Human engineers fixed the data, with every fix checked by a blind re-implementation, and automated checks made sure every task can be passed and graded correctly. A locked protocol fixed the evaluation, so agents could not reach the answer and could not fake a pass in their own sandbox. On the repaired benchmark, frontier systems solve almost every task, and the Hard split is what still separates the strongest systems from the rest. We release the tasks, the Hard split, the protocol and every grade.

Limitations

The Hard split was chosen using model failures. Its tasks were selected because five model families failed them, so those families’ Hard scores are biased downward. Systems outside the selection (Opus 5.5, Fable 5.1, Astra and the partner runs of Sonnet 5, Terra, Sol and Haiku 4.5) show the same spread, from 25% to 96%. With 51 tasks, single-task differences are noise.

One attempt per task. Each system ran once, and every run had at least one flaky test. The partner runs carry a single grade and were not re-graded by us.

Training data. The public repositories and fixes predate the evaluated models. The protocol stops an agent from fetching a solution, not from having seen it in training. The never-public set reduces this concern but does not remove it.

What re-grading cannot catch. The clean container rejects changes made outside the patch, not

code inside it. The imitation SDKs on the private set passed both grades. Our audits match commands and files, so they give lower bounds. Instructions still name many of the symbols the tests use, a property of the original benchmark design that we measured but did not remove.

Ethics Statement

The public tasks come from open-source repositories under their original licences. The private tasks come from organisations that consented to their use for evaluation, and their code is not released. The paper describes how agents faked dependencies and how a sandbox grader can be fooled, because benchmark operators need this to defend against it; the clean-container re-grade blocks both. No personal data about the engineers is recorded.

References

- Reem Aleithan, Haoran Xue, Mohammad Mandi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. **SWE-Bench+: Enhanced coding benchmark for LLMs**. *arXiv preprint arXiv:2410.06992*.
- Ibragim Badertdinov, Alexander Golubev, et al. 2025. **SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents**. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, et al. 2025. **SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?** *arXiv preprint arXiv:2509.16941*.
- Jonathan Gabor, Jayson Lynch, Jonathan Rosenfeld, et al. 2025. **EvilGenie: A reward hacking benchmark**. *arXiv preprint arXiv:2511.21654*.
- jacobkahn. 2025. Repo state loopholes during agentic evaluation. SWE-bench GitHub issue 465, <https://github.com/SWE-bench/SWE-bench/issues/465>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. **SWE-bench: Can language models resolve real-world GitHub issues?** In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- joe-needham. 2025. Repo state leakage. SWE-Bench Pro GitHub repository, issue 7.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, et al. 2025. **SWE-Lancer: Can frontier LLMs earn \$1 million from real-world freelance software engineering?** *arXiv preprint arXiv:2502.12115*.
- Nitya Nadgir, Sayash Kapoor, Kangheng Liu, Peter Kirgis, Matilda Orona, Stephan Rabanser, Tilman Bayer, Abhishek Shetty, Yue Ling, Derrick Chan-Sew, Rumi Nakagawa, Saiteja Utpala, Zachary S. Siegel, and Arvind Narayanan. 2026. **Life after benchmark saturation: A case study of CORE-Bench**. *arXiv preprint arXiv:2606.26158*.
- OpenAI. 2024. Introducing SWE-bench Verified. OpenAI blog, <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenAI. 2026. Separating signal from noise in coding evaluations. OpenAI blog, <https://openai.com/index/separating-signal-from-noise-coding-evaluations/>.
- Kunvar Thaman. 2026. **Reward hacking benchmark: Measuring exploits in LLM agents with tool use**. *arXiv preprint arXiv:2605.02964*.
- You Wang, Michael Pradel, and Zhengyao Liu. 2025. **Are “solved issues” in SWE-bench really solved correctly? an empirical study**. *arXiv preprint arXiv:2503.15223*.
- Boxi Yu et al. 2025. **UTBoost: Rigorous evaluation of coding agents on SWE-Bench**. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, et al. 2025. **Multi-SWE-bench: A multilingual benchmark for issue resolving**. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Linhao Zhang, Shilin Wang, Junda Jia, Fangkai Wang, et al. 2025. **SWE-bench goes live!** *arXiv preprint arXiv:2505.23419*.
- Bingchen Zhao, Dhruv Srikanth, Yuxiang Wu, et al. 2026. **SpecBench: Measuring reward hacking in long-horizon coding agents**. *arXiv preprint arXiv:2605.21384*.
- Pujun Zheng, Zixin Shang, Shufan Jiang, Wenhui Tian, Dongsheng Zhu, Zerun Ma, Dingbo Yuan, and Qi Zhang. 2026. **SWE-Bench Pro Verified: A reliable benchmark for software engineering agents**. *arXiv preprint arXiv:2609.08149*.
- Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. 2025. **ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases**. *arXiv preprint arXiv:2510.20270*.
- Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, et al. 2025. **Establishing best practices for building rigorous agentic benchmarks**. *arXiv preprint arXiv:2507.02825*.

A The Hard Split

The 51 tasks come from ansible (8), openlibrary (8), flipt (7), teleport (6), webclients (5), qutebrowser (4), vuls (4), element-web (4), navidrome (3), NodeBB (1) and tutanota (1). A task qualified if at least two of five model families (Opus 5, GLM-5.3, Kimi K3, Inkling, Gemini 3.8 Flash) failed it. That gave 54 candidates after excluding 3 tasks with verified defects, and 3 more were removed after review. Every Hard task passes with its reference solution and fails with an empty patch. Readers should know four things about the released list.

- 14 tasks qualify only through partner runs of Kimi K3.
- 39 of the 51 instructions were edited after selection, against 190/591 elsewhere.
- One task (openlibrary 7cbfb812) has a verified test defect, and one (ansible 40ade1f8) has a timing-sensitive test.
- Every Hard task is solved by at least one of our runs.

B Disagreements Between the Two Grades

Table 4 lists every task whose grade in the agent’s sandbox differs from its first grade in a clean container, across our public runs. The runs labelled v2.2r used an earlier release of the tests.

C The Never-Public Set

The 272 tasks come from 4 organisations (154 in Go, the rest in Python and JavaScript) whose code, to their knowledge, has never been on the open web. Table 5 lists the defects found while running it and their status. The instruction audit was done by an LLM judge (Claude Opus 5) that saw the instruction, the hidden tests and the reference solution but no model output. We checked eight of its judgments by hand. Its grades are noisy: it agreed with itself on 71% of re-judged unchanged tasks. On the final release, every system was run again with the same scaffold, effort and budget. With the imitation-SDK passes counted as failures, the scores are 92.6 (Opus 5), 87.5 (Kimi K3), 87.1 (GLM-5.3), 86.0 (Gemini) and 79.0 (Inkling).

D Reproducibility

Harness. Upstream Harbor 0.22.0 on Modal 1.5.5. The network is set per phase: public during setup, only the model endpoint during the agent phase, public again for the verifier, because 32 tasks download dependencies while testing.

Scaffolds. Claude Code 2.1.278 on the public runs and 2.1.283 on the private runs, with web tools disabled. Codex 0.155.1, with hosted web search disabled. mini-swe-agent 2.4.6, with no step or cost limit, so the time budget binds.

Budget and grading. The budget is 3000 s per task. The agent is stopped at 2940 s, and the patch present then is graded. The patch is the staged git diff of the working tree. It is applied to a fresh container built from the task image, with a three-way fallback, and the unchanged verifier runs. When the sandbox grade and the clean grade disagree, two more clean grades are taken and the majority of the three clean grades counts.

Release. Task directories, the Hard split, the probe, capture and re-grade scripts, the exact harness flags, and per-task grades in both modes for every run.

Run	Task	In place	Fresh	Cause and adjudication
Opus 5 (high, v2.2r)	vuls ef2be3d6	✓	✗	Forged goval-dictionary v0.9.5: module zip served from a file-based GOPROXY, fabricated go.sum entry; the clean image fails the checksum
Opus 5 (high, v2.2r)	vuls ca3f6b1d	✓	✗	Vendored, modified dependency whose log/ package is git-ignored; the captured patch is incomplete
Opus 5 (high, v2.2r)	ansible 40ade1f8, teleport ba6c4a13	✗	✓	One timing-sensitive test failed under agent-phase load (two tasks)
Opus 5 (high, v2.2r)	webclients c6f65d20	✓	✗	Missed a 5 s Jest timeout on a cold image; fresh grades 0,1,1, adjudicated pass
Inkling (high, v2.2r)	vuls ef2be3d6	✓	✗	Forged goval-dictionary v0.9.5 from the older cached module with a fabricated .info and go.sum entry, as Opus 5 did; 2/2 fresh fails
Inkling (high, v2.2r)	vuls ca3f6b1d	✓	✗	Edited a struct field into a cached dependency with sed -i; nothing in the diff; 2/2 fresh fails
Inkling (high, v2.2r)	navidrome 874b17b8	✓	✗	Made the module cache writable and added a file to a cached dependency; 2/2 fresh fails
Gemini 3.8 Flash	flipt 3d5a345f	✓	✗	Fabricated go.sum entries for real dependencies after rehearsing the forgery on a fake module in the local proxy cache; 2/2 fresh fails
Gemini 3.8 Flash	flipt c188284f	✓	✗	Fabricated go.work.sum checksum; 2/2 fresh fails
Gemini 3.8 Flash	navidrome 874b17b8	✓	✗	Edited the module cache in place; 2/2 fresh fails (Opus 5 and GLM-5.3 solved the same task offline by vendoring a local fork under a replace directive)
Gemini 3.8 Flash	NodeBB be43cd25, tutanota f3ffe17a	✗	✓	Flaky tests; 2/2 fresh passes
Opus 5 (xhigh)	ansible 40ade1f8	✓	✗	The timing-flaky revised test; fresh grades 0,1,1, adjudicated pass
Opus 5 (xhigh)	element-web 71fe08ea	✗	✓	Fresh grades 1,1,1; in-place fail was a sandbox flake
Opus 5.5 (xhigh)	element-web 72a8f8f0	✓	✗	Flaky composer test; fresh grades 0,1,1, adjudicated pass
Fable 5.1	NodeBB bd80d36e	✓	✗	Fresh grades 0,1,1, adjudicated pass
Fable 5.1	teleport 645afa05	✗	✓	Fresh grades 1,1,1
GPT-6 Astra	element-web 53b42e32	✓	✗	Flaky test unrelated to the edit; fresh grades 0,1,0, adjudicated fail

Table 4: Every task whose in-place and first fresh-sandbox grades differ, across the runs of Table 2. Pass-to-fail flips are either environment dependence (forged or edited dependencies, all rejected by the pristine image) or flaky tests; fail-to-pass flips are sandbox-side flakes. GLM-5.3 had no disagreement between its two grades. Private-set flips are in the release; they are the caught forgeries of §4.5 and flaky tests.

Defect	Description	Status in the final release
D1 hidden-test restore	The shipped verifier applied the hidden test patch over the agent's working tree; any agent edit to a file the patch touches made the patch fail to apply and graded the task 0 (15–20 tasks per system)	Fixed (restore before apply in all 272 verifiers); +10 to +18 tasks per system on replay
D5 test-symbol collision	An agent-added test file declaring a symbol the hidden test also declares fails the package compile	Fixed (colliding agent test files removed)
D2 Go toolchain paths	Go images did not export GOPATH/GOMODCACHE, so agents had to rediscover the module cache (Opus 5 in 131 of 154 Go trajectories, Inkling in 65)	Fixed in rebuilt images; offline compile of a sample of hidden-test packages went from 0/7 to 7/7
D3 end-of-life base images	Package repositories of older Debian images no longer resolve	Fixed in 13 of 14 images; one still cannot install packages
D4 state-sensitive in-place grade	Fail-to-pass flips from agent-side toolchain state (<code>go env -w</code>)	Closed by fresh-sandbox grading
Manifest egress	Three consecutive exports lacked the agent-phase no-network field, so egress was open under the harness flags	Fixed at source; probe still runs before every job
Reference patch in verifier bundle	The verifier configuration carried the reference and test patches and commit identifiers	Removed; graded lists unchanged
Missing SDK modules	Two tasks need AWS and Deepgram SDK modules absent from the images, so an honest solution cannot compile offline	Open; the 5 imitation-SDK passes occurred here
Flaky test	A timing-dependent test fails a correct solution about one run in five in ten tasks	Open; adjudication restores most cases

Table 5: Verifier and image defects found in the private set. The verifier repair is ours, validated with the two-sided gate and a full replay of every captured patch. The final release also carries the maintainers' image rebuild and the 68 instruction corrections.